Java

-> OOPS, class, object, oops concepts : Encapsulation, abstraction, inheritance, polymorphism

-> ArrayList vs linkedlist

ArrayList	LinkedList
Uses dynamic array	Uses doubly linked list
Fast random access	Slow random access
Insert/Delete is slower	Insert/Delete is faster
Less memory	More memory

-> hash map: A HashMap is a part of Java's Collection Framework from the java.util package that stores elements in unique key-value pairs using a data structure called a hash table. Instead of accessing data via a numerical index like an ArrayList, you use a custom key (such as a name or ID) to instantly look up its corresponding value

Not thread-safe
Allows one null key
Faster in single-threaded applications

-> ConcurrentHashmap: A ConcurrentHashMap is a highly scalable, thread-safe implementation of a hash map in Java. It allows multiple threads to read and write to the map simultaneously without blocking the entire data structure

Thread-safe
Doesn't lock the entire map
Better performance than Hashtable

-> Exception handling: Exception handling is used to handle runtime errors gracefully without terminating the application.

Keywords:

try
catch
finally
throw
throws

-> Checked Exception: Checked at compile time.

Example:

IOException
SQLException

Unchecked Exception: Occurs at runtime.

Example:

NullPointerException
ArithmeticException

-> Lambda Expressions: Lambda expressions provide a concise way to implement functional interfaces.

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello");
    }
};
```

We write

```
Runnable r = () -> System.out.println("Hello");
```

Less code
Better readability
Functional programming
Used extensively with Streams

-> Streams API: Streams API used to process collections in a functional way without modifying the original collection.

Common operations:

filter()
map()
sorted()
distinct()
collect()
reduce()

```
List<String> names =
Arrays.asList("Ram","John","Alex");

List<String> result =
names.stream()
.filter(name -> name.startsWith("A"))
.collect(Collectors.toList());
```

-> Optional: Optional is a container object that helps avoid NullPointerException.

-> Dependency Injection in Spring boot: Dependency Injection is an IoC principle where Spring creates and manages objects instead of developers creating them manually.

IOC is a more flexible way to manage object dependencies and its lifecycle (through Dependency injection). Dependency injection and IOC are the same. Dependency injection is the implementation of IOC.

IoC (Inversion of Control) in Spring Boot is a core architectural design principle that is responsible for creating, managing, and configuring application objects (beans). Using Dependency Injection

Without DI :
```
EmployeeService service =
new EmployeeService();
```

With DI:
```
@Service
public class EmployeeService {
}

@RestController
public class EmployeeController {

    @Autowired
    private EmployeeService service;
}
```

@ Component : tells Spring that you have to manage this class or bean.
@ Autowired : tells spring to resolve and add this object dependency.

Advantages:
Loose coupling
Easy testing
Better maintainability

-> Spring boot Annotations: During my Spring Boot development, I frequently used the following annotations:

Annotation	Purpose
@SpringBootApplication	Starts the Spring Boot application
@RestController	Creates REST APIs
@RequestMapping	Maps URLs
@GetMapping	GET API
@PostMapping	POST API
@PutMapping	Update API
@DeleteMapping	Delete API
@Service	Business logic layer
@Repository	Database layer

@Autowired Dependency Injection
@PathVariable Read values from URL
@RequestParam Read query parameters
@RequestBody Read JSON request
@Valid Input validation
@ExceptionHandler Handle exceptions
@ControllerAdvice Global exception handling

Interview Tip: If asked which ones you used most, say:

"I frequently used @RestController, @Service, @Autowired (or constructor injection), @GetMapping, @PostMapping, @RequestBody, @PathVariable, and @Valid while building Spring Boot REST APIs."

-> REST API flow in Spring boot:

Suppose we have:

GET /company/101

The request flows through these layers (layered architecture):

Client
↓
Controller
↓
Service
↓
Repository
↓
Database
↓
Repository
↓
Service
↓
Controller
↓
JSON Response
Responsibilities

Controller

Receives the HTTP request.
Validates inputs.
Calls the service layer.

Service

Contains business logic.
Coordinates data processing.
Calls the repository.

Repository

Interacts with the database using JPA/Hibernate.

Database

Stores and retrieves data.

Response

Controller returns a JSON response with the appropriate HTTP status code.

Example response:

```
{
  "id": 101,
  "companyName": "Backflipt",
  "industry": "Software",
  "revenue": "10M"
}
```

-> Why spring boot: Spring Boot is primarily used to simplify, accelerate, and streamline the development of production-ready Java applications. Built directly on top of the traditional Spring Framework, Spring Boot follows layered architecture reduces boilerplate configuration and provides:

Embedded Tomcat
Auto Configuration
Starter Dependencies
Easy REST API development

-> Bean: A Bean is an object managed by the Spring IoC Container.

It is created using

@Component

@Service

@Repository

@Controller

@Bean

-> REST APIs: REST APIs enable communication between frontend and backend systems using standard HTTP methods

Methods:

GET: Get resource

POST: Creates new resource

PUT: Updates the entire resource.

DELETE: Delete resource

PATCH: Updates only required fields.

Features

Stateless
JSON
HTTP
Client Server Architecture
caching

-> Handle exceptions in spring boot: Using

@ControllerAdvice

@ExceptionHandler

Create a Global Exception Handler so every API returns a consistent error response.

-> JPA: JPA (Jakarta Persistence API, formerly known as Java Persistence API) is a standard Java specification that defines how to manage and persist relational data in Java applications. It acts as an Object-Relational Mapping (ORM) bridge, allowing developers to interact with database records using standard Java objects instead of writing complex SQL queries.

Common annotations

@Entity

@Id

@GeneratedValue

@OneToMany

@ManyToOne

-> Hibernate: Hibernate is an ORM framework that maps Java objects to database tables.

Benefits

Reduces SQL
Supports JPA
Caching
Lazy Loading
Database Independence

Hibernate is one implementation of JPA.

-> JDBC: JDBC in Spring Boot refers to the framework's built-in capability to streamline database communication using the standard Java Database Connectivity (JDBC) API while eliminating the complex setup and repetitive boilerplate code traditionally required in plain Java.

-> Difference between Hibernate and JDBC:

JDBC	Hibernate
Manual SQL	Automatic ORM
More Code	Less Code
Manual Mapping	Automatic Mapping
Lower Level	Higher Level

-> Microservices architecture: Microservices architecture is an architectural pattern that divides an application into smaller independently deployable services that communicate over a network. Each service handles a specific function or responsibility of the application that improves scalability, maintainability, and deployment flexibility because each service can be developed and deployed independently.

Advantages:

Independent deployment
Better scalability
Fault isolation
Technology flexibility

-> Caching implementation: Caching stores frequently accessed data temporarily (RAM) so repeated requests can be served faster without hitting the database or external services and reduces backend load.

In our App Builder platform, we faced slow page loading because frequently used business logic was being executed on every request. After analyzing API performance, I identified repetitive backend computations as the bottleneck. I implemented a caching mechanism where results are stored after the first execution of the business logic and reused for subsequent requests with the same inputs. This significantly reduced response times by 30% and improved the overall application performance.

-> Explain one REST API you built: One of the REST APIs I developed was the Company Profile API for the IDC Knowledge Platform. This API was used to fetch detailed information about a company, such as its revenue, industry, headquarters, technology spending, employee count, and other business insights, which were displayed in the React frontend.

Example

Company Profile API

GET

/company/{id}

Flow

Controller

↓

Service

↓

Repository

↓

Database

↓

JSON Response

Mention

Validation

Exception Handling
HTTP Status Codes

-> Docker: Docker packages an application with all dependencies into containers, ensuring consistent execution across environments. Docker is used for containerization purpose.
docker run <image> – Create and start a container from an image.
docker ps – List all currently running containers.
docker ps -a – List all containers, including stopped ones.
docker stop <container_id> – Gracefully stop a running container.
docker start <container_id> – Start a stopped container.
docker restart <container_id> – Restart a container.
docker rm <container_id> – Remove a stopped container.

-> Kubernetes: Kubernetes is a container orchestration platform.

Responsibilities

Auto Scaling
Self Healing
Load Balancing
Rolling Updates

Be honest:

In my projects I primarily worked with Docker-based deployments and collaborated with DevOps teams on containerized deployments. I understand Kubernetes concepts and I'm currently strengthening my hands-on experience with deployments, pods and services.

-> Docker - Creates containers.

Kubernetes - Manages thousands of containers.

-> Explain CI/CD : CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a set of modern software development practices that automate the process of building, testing, and releasing applications.

CI - Automatically builds and tests code after commits.

CD - Automatically deploys applications to staging or production after successful validation.

Three core concepts: Continuous Integration, Continuous Delivery, Continuous Deployment

-> AWS Experience:
My hands-on experience is primarily with Amazon S3 for secure file uploads using pre-signed URLs. I also worked with Docker and CI/CD pipelines in projects deployed to cloud environments. While I haven't provisioned AWS infrastructure myself, I understand the purpose of services such as:

Amazon S3 – Object storage for files
EC2 – Virtual servers
IAM – User and permission management
CloudWatch – Monitoring and logging
ECR – Docker image repository

My main contribution has been integrating applications with AWS services rather than managing the cloud infrastructure.

My primary AWS experience is with Amazon S3 for secure file storage and uploads. In one of my projects, we implemented multipart file uploads where users could upload large files efficiently.

Instead of sending files through our backend server, we used AWS S3 pre-signed URLs. The backend generated a temporary, secure URL with limited permissions and an expiration time. The frontend used this URL to upload files directly to S3.

This approach reduced the load on our backend server, improved upload performance, and enhanced security because AWS credentials were never exposed to the client.

I worked on integrating this functionality into the backend REST APIs and coordinated with the frontend to complete the upload flow.

-> Authentication is the process of verifying the identity of a user or system. It ensures that the user is legitimate by validating credentials like passwords, OTPs, or biometrics.

-> Authorization determines the access rights and permissions of an authenticated user. It decides what resources the user can access and what actions they are allowed to perform.

-> Why do you want to join WaferWire: WaferWire works on enterprise applications, cloud-native modernization and Java integration technologies, which align well with my experience in Spring Boot, React, REST APIs and AWS. I'm particularly interested in expanding my expertise in enterprise integration, Kubernetes and cloud-native development while contributing to large-scale customer applications.

1. Explain the architecture of your current project.

This is probably the first question.

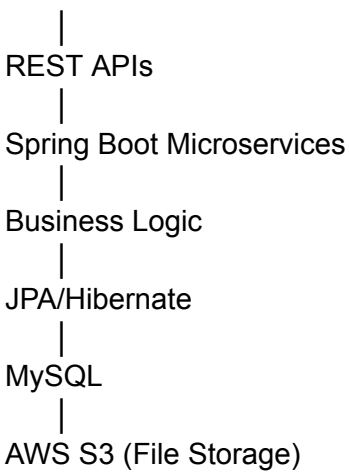
Interview Answer

In my recent project, the frontend was developed using React.js and communicated with Spring Boot microservices through REST APIs.

The React application handled the UI, routing, and API integration. The backend consisted of multiple Spring Boot microservices responsible for different business functionalities, such as company profiles and analytics. These services interacted with the database using Spring Data JPA/Hibernate.

Frequently used business logic was optimized using caching to improve performance. Applications were containerized using Docker and deployed through CI/CD pipelines. For file uploads, we integrated Amazon S3 using pre-signed URLs.

Architecture:



Question 2
What happens when a user opens your application?

This checks your end-to-end understanding.

Interview Answer

When a user opens the application, the React application loads in the browser. Based on the route, the required React component is rendered.

The component calls the backend REST API using Axios or Fetch.

The Spring Boot Controller receives the request, passes it to the Service layer, which performs business logic and fetches data from the Repository.

The Repository retrieves data from the database, and the response is returned as JSON to React, where it is displayed to the user.

Question 3
How did you improve application performance?

Very likely because your resume mentions performance improvements.

Interview Answer

I focused on both frontend and backend optimizations.

On the frontend, I created reusable React components, optimized rendering, reduced unnecessary re-renders, and improved routing performance.

On the backend, I implemented caching for frequently accessed business logic, reducing repeated computations and database calls, which improved API response times by around 30%.

We also monitored code quality using SonarQube and optimized SQL queries where required.

Question 4
Explain Docker and how you used it.
Interview Answer

Docker is a containerization platform that packages an application along with its dependencies into a container, ensuring consistent execution across different environments.

In my project, Docker was used to containerize backend services, allowing developers to run the same application locally, in testing, and in production without environment-specific issues.

We built Docker images, ran containers, and integrated Docker into our CI/CD pipeline for deployments.

Question 5
What is Kubernetes? Did you work on it?

The JD emphasizes Kubernetes.

Interview Answer

My direct hands-on experience is primarily with Dockerized applications. I have worked closely with applications that were deployed through CI/CD pipelines, while Kubernetes deployments were managed by the DevOps team.

I understand Kubernetes concepts such as Pods, Deployments, Services, ReplicaSets, ConfigMaps, and auto-scaling. Kubernetes orchestrates containers, provides self-healing, rolling updates, and load balancing.

Although I haven't independently managed Kubernetes clusters, I understand how containerized Spring Boot applications are deployed and managed in Kubernetes environments.

This is an honest answer and much better than pretending deep Kubernetes experience.

Question 6
What are Microservices? What challenges did you face?
Interview Answer

Microservices divide a large application into smaller, independently deployable services.

In our project, different business functionalities were developed as separate services, allowing teams to develop and deploy them independently.

One challenge was maintaining communication between services and ensuring consistent API contracts. We addressed this through well-defined REST APIs, proper documentation, and regular integration testing.

Question 7
What happens if one microservice is down?

Very common interview question.

Interview Answer

If one microservice becomes unavailable, only the functionality handled by that service is affected, while the rest of the application continues to operate.

In production, common practices include implementing retries, circuit breakers, fallback mechanisms, logging, monitoring, and health checks. These improve resilience and help isolate failures without bringing down the entire system.

Question 8
You don't have Apache Camel experience. What do you know about it?

This is highly likely because it's in the JD.

Interview Answer

I haven't worked directly on Apache Camel in a production project, but I understand its purpose.

Apache Camel is an enterprise integration framework that simplifies communication between different applications using predefined Enterprise Integration Patterns.

It supports routing, transformation, filtering, and protocol conversion between systems.

Since my work has primarily involved REST API integrations, I believe my experience with service integration provides a solid foundation for learning Apache Camel quickly.

Do not claim hands-on experience if you don't have it.

Question 9
Describe a production issue you solved.

Behavioral + technical.

Interview Answer

One issue involved slow API responses for frequently accessed business data.

After analyzing the application flow, we found that the same business logic and database queries were executed repeatedly.

I helped optimize this by introducing caching for frequently requested data, reducing redundant processing and improving API response time by approximately 30%.

After testing the changes and validating performance improvements, we deployed the solution through the standard CI/CD pipeline.

Question 10
Why should we hire you?

Usually the last technical question.

Interview Answer

I have over two years of experience building enterprise web applications using React.js and Spring Boot. I have worked on REST APIs, microservices, performance optimization, Dockerized applications, AWS S3 integration, and Agile development practices.

While I don't have production experience with Apache Camel or Kubernetes administration, I have a strong foundation in Java and backend development, and I learn new technologies quickly.

I believe I can contribute immediately in areas such as Spring Boot, REST APIs, React, and enterprise application development while continuing to grow into the integration and cloud-native technologies your team uses.

--> Hi, I'm Vasavi. I'm a Full Stack Developer with 2+ years experience building web applications using React.js, Next.js, Node.js, Express.js, Java, Spring boot, Python, MongoDB, and MySQL.
My experience includes contributing to scalable web applications, building reusable UI components, developing RESTful APIs,integrating frontend with backend services, optimizing application performance, debugging, and delivering end-to-end features across the full stack application development in Agile environments. I have also worked extensively with Git-based workflows, code reviews.

--> I've worked on projects such as Appbuilder platform (low-code application) that enables users to create and customize dynamic applications through an interactive drag-and-drop interface
The platform allows users to visually design pages by adding components such as rows, columns, tables, headers, sliders, style classes, custom widgets, business logics for app operations while automatically generating and managing the underlying configuration. And

also provides Dynamic Layout Management with support for nested components, Role-Based Access Control (RBAC) for users, and library support across applications

and also worked on velocity sales (a saas application) built on top international data corporation data, building dashboards, creating market assessments between tech leaders and providing analytics for tech spenders and buyers based on their market value, geography,...

Worked Velocity sales a SaaS-based sales intelligence and analytics platform built on top of data from International Data Corporation (IDC). I worked on developing dashboards and data-driven reporting features that helped technology vendors and buyers analyze market opportunities

Explain a challenging bug you solved :
-> In our App Builder platform, we faced slow page loading because frequently used business logic was being executed on every request. After analyzing API performance, I identified repetitive backend computations as the bottleneck. I implemented a caching mechanism where results are stored after the first execution of the business logic and reused for subsequent requests with the same inputs. This significantly reduced response times and improved the overall application performance.

How did you decide what to cache?
A: We identified business logic that was called frequently and produced the same output for identical inputs, especially those involving expensive database queries or computations.

What caching technology did you use?
A: Depending on the environment, we used an in-memory cache mechanism (or Redis if applicable) to store and retrieve results efficiently.

How did you handle stale data?
A: We implemented cache invalidation by setting expiration times (TTL) and clearing or refreshing cache entries whenever the underlying data was updated.

Explain a feature you owned end-to-end :
I owned the implementation of multipart file upload feature using AWS S3 pre-signed URLs. When a user uploaded a file, it was split into chunks and each chunk received a pre-signed URL for direct upload to S3 within expirable time. So, I built Express APIs to generate AWS S3 pre-signed URLs for file chunks. I also implemented APIs to regenerate pre-signed URLs for specific chunks if they are expired, allowing uploads to resume without restarting the entire file upload. This improved upload reliability and user experience for large files.

Reduces backend load.
Files go directly from browser to S3.
Better scalability and lower server bandwidth costs.

Used the browser's upload progress events for each chunk.

Implemented retry logic.
If the URL expired, requested a new pre-signed URL for only the failed chunk.

4. How do React and Node.js communicate?

Expected answer: React communicates with Node.js(backend services) through REST APIs. The frontend sends HTTP requests using fetch or Axios, and the Express backend processes requests, interacts with databases or external services, and returns JSON responses that are rendered by the UI.

5. What happens when a user enters a URL in the browser?
Very common AI interview question.

Short answer: Browser performs DNS lookup, establishes connection with the server, sends HTTP request, backend processes it, database is queried if required, response is returned, and browser renders the page.

8. What is Node.js?
Expected:Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run outside the browser. It uses an event-driven, non-blocking I/O model.

9. What is Express.js?
Express.js is a lightweight web framework for Node.js used to build REST APIs and backend applications.

What is Middleware?

Example: app.use(authMiddleware);

Answer:
Middleware functions execute before the request reaches the route handler. They are commonly used for authentication, logging, validation, and error handling.

What is JWT?
Answer: JWT(JSON Web Token) is a token-based authentication mechanism. After login, the server generates a signed token which is sent by the client in subsequent requests for user verification.

What is MongoDB?
MongoDB is a NoSQL document database that stores data in JSON-like BSON documents.
Flexible schema
Fast development
Horizontal scaling

What is Redis?
Answer: Redis is an in-memory key-value store commonly used for caching, session storage, rate limiting, and improving application performance.

Authentication is the process of verifying the identity of a user or system. It ensures that the user is legitimate by validating credentials like passwords, OTPs, or biometrics.

Authorization determines the access rights and permissions of an authenticated user. It decides what resources the user can access and what actions they are allowed to perform.

Caching stores frequently accessed data temporarily (RAM) so repeated requests can be served faster without hitting the database or external services and reduces backend load.

Why did you use AWS S3 Presigned URLs?
Answer: Presigned URLs allow users to upload files directly to S3 without exposing AWS credentials, reducing backend load and improving scalability, and provides secure access with configurable expiration times.

Why Multipart Upload?
Answer: Multipart uploads improve reliability and performance for large files because chunks can be uploaded independently and retried if failures occur.

How do you improve application performance? (80% chance)
Answer: I improve performance through caching, API optimization, query optimization, lazy loading, code splitting, and reusable component design. In one project, implementing caching reduced API response times by approximately 30%.

What are REST APIs and why are they important? (75% chance)
Answer: REST APIs enable communication between frontend and backend systems using standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE. They help create scalable, loosely coupled applications and support system integrations.

What is caching and where have you used it? (70% chance)
Answer: Caching stores frequently accessed data temporarily to reduce repeated processing and database calls. I implemented caching in business logic layers, which reduced response times and improved overall application performance.

Explain microservices architecture (65% chance)
Answer: Microservices architecture is an architectural pattern that divides an application into smaller independently deployable services that communicate over a network. Each service handles a specific function or responsibility of the application that improves scalability,

maintainability, and deployment flexibility because each service can be developed and deployed independently.

Tell me about a challenging problem you solved (60% chance)
Answer: While implementing multipart uploads, I had to handle cases where presigned URLs expired before uploads completed. I designed a mechanism to regenerate URLs only for failed chunks, improving reliability while avoiding full upload restarts.